

Guia da biblioteca smosh

Explicado em linguagem simples

Um passeio guiado pelo que a biblioteca `smosh` faz, como suas partes se encaixam, e o que vale a pena arrumar antes de construir algo em cima dela.

Documento gerado a partir da leitura do código-fonte

1. Antes de tudo: o que é isso?

O **smosh** é um pedaço de código Python criado para facilitar o uso de outro programa, chamado **Modeller**. O Modeller é um software científico usado por pesquisadores de biologia para descobrir a "forma 3D" de uma proteína a partir da sua sequência (a lista de aminoácidos que a compõe) — algo como montar um quebra-cabeça 3D usando como referência quebra-cabeças parecidos que já foram montados antes.

PENSANDO DE FORMA SIMPLES

O Modeller é como um artesão especialista que sabe montar réplicas de esculturas a partir de uma receita muito específica e complicada de escrever. O **smosh** é o assistente que escreve essa receita para você, entrega ao artesão, espera ele terminar, e organiza o resultado numa pasta arrumada — para que você não precise aprender a "falar a língua" do artesão.

Ou seja: o smosh não faz a modelagem da proteína sozinho — quem faz isso é o Modeller. O papel do smosh é **automatizar tudo em volta**: gerar o script certo para cada etapa do processo, rodá-lo, capturar os erros e organizar os arquivos de resultado.

2. A ideia geral: uma linha de montagem

O processo de "descobrir a forma de uma proteína" não acontece de uma vez só — ele é dividido em várias etapas, uma dependendo do resultado da anterior, como uma linha de montagem numa fábrica. O smosh representa cada etapa dessa linha de montagem como uma "caixinha" de código (uma classe Python), e todas essas caixinhas seguem o mesmo formato por dentro, o que facilita adicionar novas etapas no futuro.

Linha de montagem típica de uma modelagem:

1. Pegar a sequência da proteína a partir de um arquivo PDB (estrutura já conhecida)

↓

2. Buscar "modelos parecidos" (templates) num banco de dados de estruturas já resolvidas

↓

3. Alinhar a sequência nova com o template encontrado

↓

4. Construir o modelo 3D da proteína, baseado nesse alinhamento

↓

5. (Opcional) Refinar pedaços específicos do modelo (loops)

↓

6. Avaliar a qualidade do modelo gerado, comparando com o template

Cada uma dessas seis etapas vira uma classe própria dentro do smosh, todas seguindo o mesmo "molde" — o que em programação se chama de **classe base**. Explico esse molde a seguir, porque entender ele ajuda a entender todo o resto.

3. O "molde" que toda etapa segue

Existe um arquivo central, chamado `ModelingStep.py`, que define o comportamento comum a todas as etapas da linha de montagem. Pense nele como um formulário padrão que toda etapa preenche, com quatro perguntas fixas:

1. Quais arquivos eu preciso receber?

Cada etapa recebe um ou mais arquivos de entrada (por exemplo, um PDB ou uma sequência). Antes de começar, o smosh copia esses arquivos para uma pasta temporária só daquela execução — como separar os ingredientes numa bancada limpa antes de cozinhar.

2. Qual "receita" eu mando o Modeller seguir?

Cada etapa monta, como texto, um script Python específico que o Modeller entende — é literalmente uma receita escrita na hora, com os nomes de arquivo certos já preenchidos.

3. Rodar e conferir se deu certo

O script é entregue ao Modeller para executar. Se algo der errado, o smosh lê o registro (log) da execução e avisa com uma mensagem de erro própria (chamada `ModellerException`), carregando o conteúdo desse log — como avisar "o forno apresentou X erro, aqui está o registro completo".

4. Quais arquivos eu devo entregar de volta?

Terminada a execução, os arquivos de resultado são copiados da pasta temporária de volta para a pasta onde o usuário vai procurá-los — como tirar o prato pronto do forno e levar até a mesa.

POR QUE ISSO É UMA BOA NOTÍCIA

Como todas as etapas seguem o mesmo molde, adicionar uma nova etapa no futuro (por exemplo, um novo tipo de refinamento) exige escrever bem menos código do que se cada etapa fosse feita do zero — só é preciso descrever a "receita" específica daquela etapa; o resto (copiar arquivos, rodar, tratar erro) já vem pronto.

4. As seis etapas, uma por uma

4.1 Pegar a sequência a partir de um PDB

Classe: `GetSequenceFromPDBStep`

Recebe um arquivo PDB (um formato de arquivo que descreve a posição de cada átomo de uma estrutura de proteína já conhecida) e extrai dele a sequência de aminoácidos, salvando num formato mais simples chamado PIR — que é basicamente um "texto puro" da sequência.

4.2 Buscar templates parecidos

Classe: `FindTemplatesStep`

Recebe a sequência da proteína que se quer modelar e a compara contra um grande banco de sequências já conhecidas (chamado `pdb_95`, que já vem junto com o smosh), procurando as que mais se parecem. É o passo de "encontrar a peça de referência" para montar o quebra-cabeça.

PONTO DE ATENÇÃO

Esta etapa depende de um caminho de pasta fixo no código (explico melhor na seção 7), o que significa que, se o smosh for movido para outro computador, essa etapa pode parar de achar o banco de dados até alguém corrigir esse caminho manualmente.

4.3 Alinhar as sequências

Classe: `AlignStep`

Pega a sequência da proteína alvo e o template encontrado, e os alinha — ou seja, organiza as duas sequências lado a lado, casando os trechos parecidos, como alinhar duas frases parecidas em idiomas diferentes para ver onde cada palavra corresponde.

4.4 Construir o modelo 3D

Classe: `BuildModelStep`

Com o alinhamento pronto, esta etapa manda o Modeller efetivamente construir a estrutura 3D da proteína alvo, usando o template como referência. Por padrão são gerados **5 modelos diferentes** (pequenas variações), cada um com duas notas de qualidade calculadas automaticamente (chamadas DOPE e GA341) — como pedir 5 esculturas ligeiramente diferentes e uma nota de qualidade para cada uma.

4.5 Refinar um pedaço específico (loop)

Classe: `LoopRefinementStep`

Às vezes uma parte específica do modelo (um "loop", um trecho mais flexível da proteína) fica com qualidade ruim e vale a pena refazer só aquele pedaço, sem reconstruir tudo de novo. Essa etapa recebe o modelo já pronto e o intervalo de aminoácidos que devem ser refeitos, e gera 5 novas versões só daquele trecho.

4.6 Avaliar a qualidade do resultado

Classes: `MakeProfile`, `EvaluateEnergy` e `EvaluateEnergyofLoopRefinementStep`

Essas três classes trabalham juntas para gerar um **gráfico comparativo** entre a qualidade do template original e a do modelo construído (e, se houver, também da versão com o loop refinado) — um jeito visual de conferir se o modelo final ficou parecido em qualidade com a estrutura de referência.

5. As ferramentas de apoio (fora da linha de montagem)

Além da linha de montagem principal, o smosh tem um segundo grupo de "ferramentas de bolso" que ajudam a mexer nos arquivos de proteína sem precisar do Modeller — são cálculos e edições de texto simples, feitos em Python puro.

Ferramenta	O que ela faz, em uma frase
<code>PDBFile</code>	Lê um arquivo PDB e permite consultar quais cadeias e "peças extras" (heteroátomos) ele tem, além de remover moléculas de água.
<code>pdb</code>	Uma versão mais antiga, com o mesmo propósito do <code>PDBFile</code> , mas com nomes de método em português.
<code>AlignFile</code>	Lê e edita um arquivo de alinhamento, permitindo trocar "peças extras" de lugar entre sequências.
<code>PirSequence</code>	Representa uma única sequência dentro de um arquivo de alinhamento — usada por dentro pelo <code>AlignFile</code> .
<code>TemplateProfile</code>	Lê a lista de templates encontrados na etapa 4.2 e escolhe automaticamente o de maior semelhança.
<code>GetDataFromPDB</code>	Baixa um arquivo PDB direto do site oficial (RCSB), caso ele ainda não exista numa pasta local.

PONTO DE ATENÇÃO

Repare que existem **duas** ferramentas com o mesmo objetivo: `PDBFile` e `pdb`. Isso normalmente acontece quando uma versão mais nova é criada, mas a mais antiga não é removida — o que pode confundir quem for usar a biblioteca no futuro, sem saber qual das duas é "a certa".

6. Quando algo dá errado

Sempre que o Modeller falha ao executar uma etapa, o smosh levanta um erro chamado `ModellerException`. Esse erro carrega dentro dele o texto completo do registro de execução (o "log"), então quem programa em cima do smosh consegue ver exatamente o que o Modeller reportou.

EM OUTRAS PALAVRAS

Existe apenas **um tipo** de erro para tudo que pode dar errado — seja um template que não foi encontrado, um arquivo mal formatado, ou uma falha de configuração. Não há como, só olhando o tipo do erro, saber automaticamente qual foi a causa; é preciso ler o texto do log dentro dele.

7. Testes: como se sabe que o smosh funciona

O smosh tem uma **suíte de testes automatizados** — pequenos programas que rodam sozinhos e conferem se cada etapa se comporta como esperado (por exemplo: "depois de rodar, o arquivo de saída realmente aparece na pasta certa?"). Isso é escrito com duas ferramentas:

- **unittest**: a ferramenta de testes que já vem embutida no Python.
- **should_dsl**: uma ferramenta extra que deixa a forma de escrever as conferências mais parecida com uma frase em inglês (por exemplo, `resultado |should| equal_to(esperado)`, algo como "o resultado deveria ser igual a...").

Além dos testes individuais de cada etapa, existe um teste maior, chamado teste de **aceitação**, que roda o processo inteiro do início ao fim — da busca de templates até o gráfico final de qualidade — simulando um uso real da biblioteca.

PONTO DE ATENÇÃO

Esses testes só funcionam num computador que já tenha o Modeller instalado (com licença válida) e um caminho de pasta específico configurado — então não é possível simplesmente "baixar e rodar os testes" em qualquer máquina sem preparar esse ambiente antes.

8. O que é preciso ter instalado para o smosh funcionar

Não existe, no material enviado, uma lista pronta de dependências (o que normalmente ficaria num arquivo chamado `requirements.txt`). A lista abaixo foi montada a partir da leitura do código:

O que é necessário	Para quê
Modeller (com licença acadêmica)	É o programa que efetivamente faz a modelagem — o smosh só o comanda.
Um Python mais antigo (versão 2)	O Modeller, nesta configuração, é chamado a partir desse interpretador mais antigo — separado do Python usado pelo resto do smosh.
Python (versão 3)	É a linguagem usada para escrever o restante da biblioteca smosh.
matplotlib e numpy	Bibliotecas usadas para desenhar o gráfico de qualidade do modelo.
should_dsl	Usada apenas para rodar os testes automatizados.

PENSANDO DE FORMA SIMPLES

É como se o smosh precisasse de duas cozinhas ligadas: uma mais moderna (Python 3), onde a maior parte do trabalho de organização acontece, e uma mais antiga (Python 2), que é a única que sabe conversar com o forno especial do Modeller. As duas precisam estar disponíveis ao mesmo tempo para tudo funcionar.

9. Resumo dos pontos que merecem atenção

Reunindo tudo que foi observado ao longo do código, esta é a lista de coisas que valeria a pena revisar antes de construir algo novo (como um site ou uma interface) por cima do smosh:

1. **Caminho de pasta fixo no código.** Em vez de o smosh descobrir sozinho onde está instalado, esse caminho está escrito diretamente no código — como um endereço de casa escrito à mão numa etiqueta, que só funciona se a "casa" nunca mudar de lugar.
2. **Duas versões de Python ao mesmo tempo.** Uma parte roda em Python 3, mas a execução do Modeller em si ainda depende de Python 2 — uma versão mais antiga da linguagem, que precisa continuar instalada só para essa finalidade.
3. **Trechos de código ainda escritos no estilo antigo (Python 2)** em duas das classes de avaliação de energia e numa das ferramentas de apoio — o que pode causar erro se um dia essas partes forem executadas diretamente em Python 3.
4. **Duas ferramentas fazendo a mesma coisa** (`pdb` e `PDBFile`), além de uma pasta inteira antiga (chamada `ModelingTools.old`) que parece ter sido deixada para trás — vale confirmar se ainda é usada em algum lugar ou se pode ser removida.
5. **Um possível erro de lógica** na função que remove moléculas de água de um arquivo PDB — o teste usado no código parece manter o resultado oposto ao que o nome da função promete.

6. **Um único tipo de erro para tudo que pode falhar**, sem diferenciar as causas — o que dificulta tratar cada tipo de problema de forma diferente automaticamente.
7. **Falta uma lista oficial de dependências** — hoje é preciso ler o código inteiro (como foi feito aqui) para descobrir tudo que precisa estar instalado.

Este guia foi escrito a partir da leitura direta do código-fonte enviado (`smosh_tar.gz`), com explicações simplificadas e analogias para facilitar o entendimento por quem não é da área de programação ou biologia computacional. Havendo interesse, cada um dos pontos de atenção acima pode ser aprofundado, ou o assunto pode seguir para o desenho de uma interface (por exemplo, em Django) por cima desta biblioteca.